DEVOPS – COMPLETE EXAM REVISION SHEET (All Commands + Codes + Schemas)

This sheet merges: DevOps intro, Jenkins, Docker, Kubernetes, SonarQube, Continuous Monitoring.

1) DevOps & CI/CD

Definition: DevOps is a culture and set of practices that connects Dev and Ops to deliver software faster and more reliably.

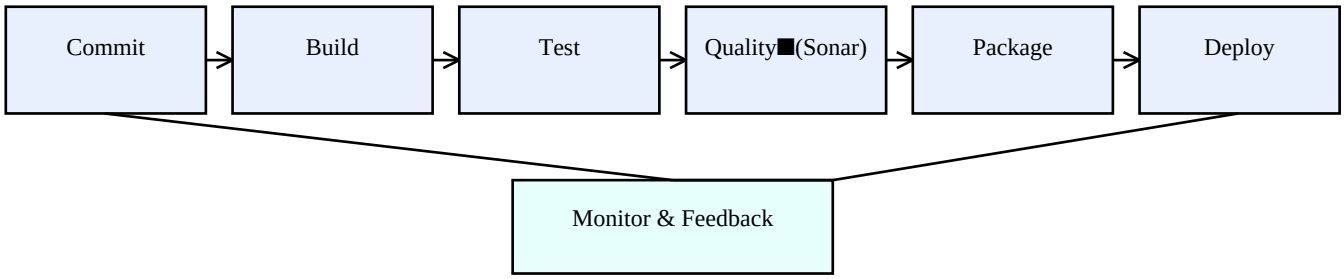
Why DevOps?

- Dev and Ops worked in silos → slow releases.
- Manual deployments → risky, error-prone.
- Need automation + fast feedback loops.

CI vs Continuous Delivery vs Continuous Deployment

- **CI**: every commit triggers **build + tests** automatically (detect bugs early).
- **Continuous Delivery**: auto deploy to test/preprod environments; **production is manual approval**.
- **Continuous Deployment**: if pipeline passes, **deploy to production automatically**.

CI/CD loop (draw)




```
Key exam sentence:
CI = integrate often + automatic build/test.
CDelivery = always ready to release (manual prod).
CDeployment = automatic prod deployment.
```

2) Jenkins (CI Pipelines)

Jenkins is an automation server used to run CI/CD pipelines (build, test, quality, deploy).

Common triggers

- Manual build
- Webhook after Git push/merge
- Scheduled build (cron)

Enable Jenkins at boot

```
sudo systemctl enable jenkins
```

Declarative Jenkinsfile (must know)

```
pipeline {
  agent any
  stages {
    stage('Checkout') {
      steps { git url: 'https://github.com/user/repo.git', branch: 'main' }
    }
    stage('Build') { steps { sh 'mvn clean compile' } }
    stage('Test')  { steps { sh 'mvn test' } }
  }
  post {
    success { echo 'SUCCESS' }
    failure { echo 'FAIL' }
  }
}
```

Sonar stage inside Jenkinsfile

```
stage('SonarQube Analysis') {
  steps {
    withSonarQubeEnv('MySonarServer') {
      sh 'mvn sonar:sonar -Dsonar.projectKey=MyProject'
    }
  }
}
```

How to explain pipeline quickly in exam

Pipeline = stages. Each stage runs steps. If a stage fails, pipeline stops. Post section runs actions on success/failure.

3) Docker

Containerization runs apps with their dependencies in lightweight containers (share host OS kernel).

Key concepts

- **Image**: read-only template.
- **Container**: running instance of an image.
- **Registry**: store images (Docker Hub).

Most asked Docker commands

```
docker pull nginx:1.15.12
docker images
docker run nginx
docker run -it alpine sh
docker ps
docker ps -a
docker exec -it <container_id> sh
docker rm <container_id>
docker rmi <image_id>
```

Dockerfile (example Spring Boot JAR)

```
FROM eclipse-temurin:17-jdk
WORKDIR /app
ADD target/app.jar app.jar
EXPOSE 8080
CMD ["java","-jar","app.jar"]
```

Build / tag / push

```
docker build -t user/myapp:1.0 .
docker tag myapp user/myapp:1.0
docker login --username=user
docker push user/myapp:1.0
```

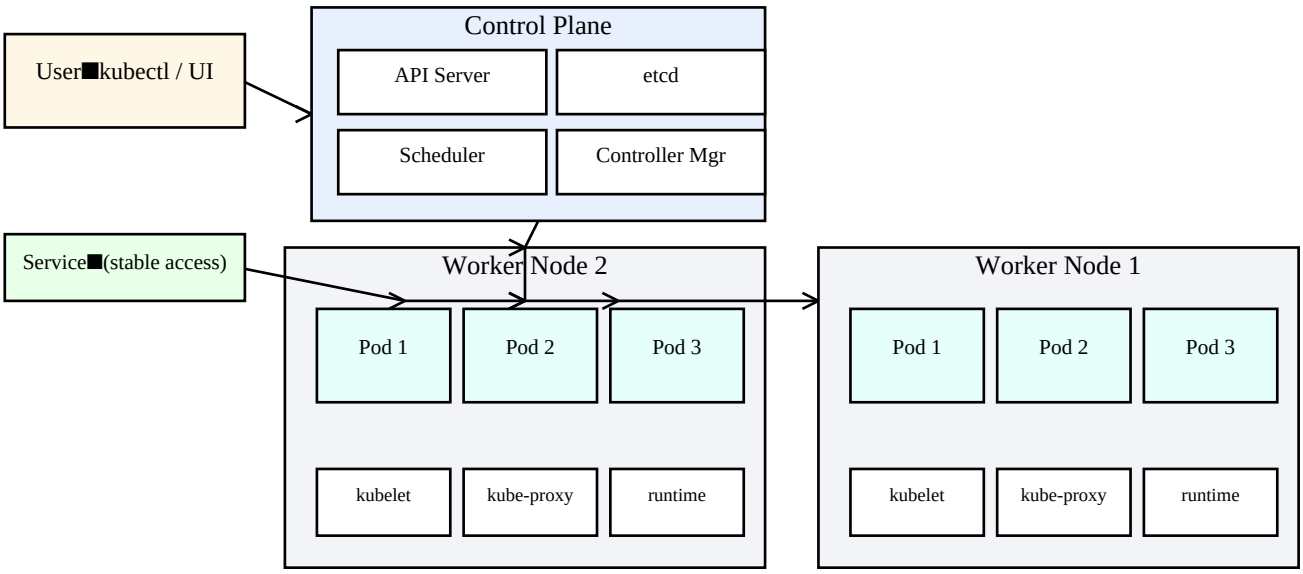
Exam tip

CMD can be overridden at runtime: **docker run myimage echo hello**. ENTRYPOINT is different (fixed entry).

4) Kubernetes

Kubernetes orchestrates containers: **self-healing, scaling, rolling updates, networking, storage**.

Architecture (draw)



Control Plane components

- **API Server**: entry point, validates requests, exposes REST API.
- **etcd**: key-value DB storing cluster state.
- **Scheduler**: chooses which node runs a Pod.
- **Controller Manager**: keeps desired state = current state.

Worker Node components

- **kubelet**: node agent; ensures Pods run as requested.
- **kube-proxy**: service networking / routing.
- **container runtime**: runs containers (containerd/CRI-O).

Main objects

- **Pod**: smallest unit (1+ containers).
- **Deployment**: manages ReplicaSets, rolling updates, scaling.
- **ReplicaSet**: maintains N replicas of Pods.
- **Service**: stable IP/DNS + load balancing to Pods (selectors).
- **ConfigMap**/**Secret**: configuration & credentials.
- **PVC**: persistent storage request.

Must-know kubectl commands

```
kubectl create ns monitoring
kubectl apply -f file.yaml
kubectl get pods
kubectl get deploy
kubectl get svc
kubectl describe pod <pod>
kubectl logs <pod>
kubectl describe svc <service>
```

```
kubectl get endpoints <service>   # super important for 503
```

Rolling update + rollback

```
kubectl set image deployment/myapp-deploy myapp=nginx:1.26
kubectl rollout undo deployment/myapp-deploy
```

Minimal YAML (Deployment + Service)

```
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx
        ports:
        - containerPort: 80

---
# service.yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
spec:
  type: ClusterIP
  selector:
    app: nginx
  ports:
  - port: 8080
    targetPort: 80
```

Service types

- **ClusterIP**: internal only.
- **NodePort**: external via NodeIP:NodePort.
- **LoadBalancer**: cloud public IP.

5) SonarQube (Static Code Analysis)

SonarQube performs **static analysis**: bugs, vulnerabilities, code smells, duplications, complexity, and quality gates.

Run SonarQube with Docker

```
docker run -d -p 9000:9000 sonarqube:8.9.7-community
```

Open: http://IP:9000 — default login: admin/admin (then change password).

Jenkins integration stage

```
stage('SonarQube Analysis') {
  steps {
    withSonarQubeEnv('MySonarServer') {
      sh 'mvn sonar:sonar -Dsonar.projectKey=MyProject'
    }
  }
}
```

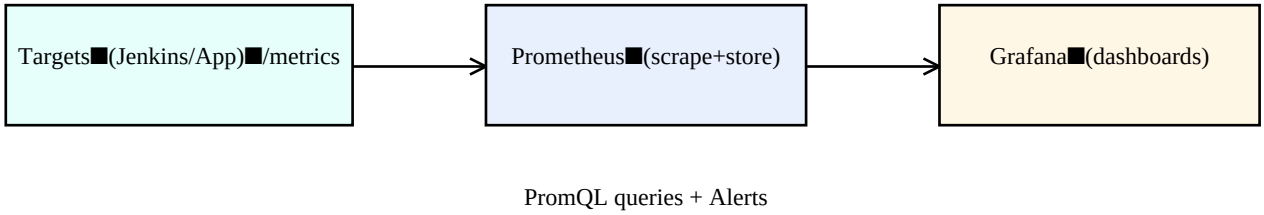
Explain in exam

SonarQube = static test (without running the app). It enforces quality rules and produces dashboards + Quality Gates.

6) Continuous Monitoring (Prometheus + Grafana)

Monitoring ensures availability and performance in production. Prometheus collects metrics; Grafana visualizes dashboards.

Monitoring architecture (draw)



Run Prometheus + Grafana (Docker)

```
# Prometheus
docker run -d --name prometheus -p 9090:9090 prometheus

# Grafana
docker run -d --name grafana -p 3000:3000 grafana/grafana
```

Useful URLs

- **Prometheus UI**: http://IP:9090
- **Prometheus Targets**: http://IP:9090/targets
- **Grafana UI**: http://IP:3000 (admin/admin first login)

Prometheus config check

```
docker exec prometheus cat /etc/prometheus/prometheus.yml
```

Quick exam answers

- 503 while Pod Running? check Service + Endpoints: **kubectl describe svc** and **kubectl get endpoints**.
- Docker show all containers: **docker ps -a**.
- Exec inside running container: **docker exec -it <id> sh**.
- Expose deployment to cluster: **kubectl expose deployment ... --type=ClusterIP**.